

READY-TO-USE PROMPT / 01

OpenBEXI Timeline illustrated rebuild brief

Reverse engineer the complete application. Preserve its capabilities, improve its usability and performance, and deliver a thoroughly verified implementation.

Role and mission

Act as a senior reverse-engineering specialist, full-stack architect, interactive timeline developer, UX designer, and software quality engineer. Analyze the repository below, then implement an improved version of OpenBEXI Timeline:

https://github.com/arcazj/openbexi_timeline

Preserve the application's verified functionality and data compatibility while making it faster, more reliable, easier to maintain, and more intuitive. Provide both an interactive **Gantt timeline** and an equivalent **tabular view** of the same data. Write all authored project documentation in **Markdown**, with a simple, practical README.md.

1. Establish the complete baseline before redesigning

- Obtain the repository and record the branch and exact commit SHA. Inventory all tracked files. Read every first-party source file and relevant configuration, test, example, and documentation file. Classify generated, vendored, and binary files; explain exclusions. Record inaccessible or unreviewed material explicitly.
- Inspect frontend and backend code, rendering and interaction logic, data models, persistence, APIs, integrations, dependencies, build scripts, and deployment. Begin with README.md, package.json, pom.xml, the HTML entry points, Dockerfile, launch scripts, and the source, tests, data, configuration, and documentation directories actually present.
- Run the original application in a reproducible local environment when possible. Exercise its screens and workflows, inspect representative datasets, and capture baseline behavior, screenshots, errors, and performance. If execution is blocked, document the reason and distinguish source-derived conclusions from observed behavior.
- Create a feature inventory and traceability matrix linking each capability to source locations, current behavior, replacement behavior, and verification. Separate intentional behavior, defects, incomplete features, and proposed enhancements. Do not infer an active feature merely from a dependency declaration.

Visual priority: preserve the legacy Gantt appearance. The illustrated visual contract later in this prompt defines the required look of both views.

Working rule: complete discovery before committing to the architecture. Continue through implementation, validation, and documentation; keep findings and progress in Markdown. Resolve routine choices autonomously and document assumptions.

Architecture and compatibility

Use the original application as the functional reference and the evidence from the audit to choose the implementation.

2. Design a maintainable replacement

- Explain the current architecture, data flow, module responsibilities, and main weaknesses. Compare a focused refactor with a broader rebuild. Select the simplest approach that meets the requirements and document the decision and tradeoffs.
- Evaluate the existing JavaScript rendering stack and Java build/backend components before replacing them. Verify actual runtime use. Choose maintained dependencies and reproducible versions; justify any framework, renderer, database, or protocol change with compatibility, performance, and maintenance evidence.
- Separate domain data and time calculations from rendering, view state, data access, and persistence. Make timeline and table components consume shared operations and selectors. Avoid duplicate business rules, uncontrolled global state, circular dependencies, and unnecessary services.
- Keep the original implementation available for comparison. Work in an isolated branch or project directory and deliver runnable increments: baseline and audit, shared model, timeline, table, integration, optimization, tests, and documentation.

3. Preserve data and integration contracts

- Document actual event identifiers, date formats, durations, groups, metadata, configuration, filters, API contracts, persistence rules, and live-update mechanisms. Preserve all verified capabilities and supported inputs, or supply a documented, tested compatibility adapter or migration.
- Use stable record identifiers and one authoritative domain model. Define validation, required and optional fields, null values, date precision, interval boundaries, timezone conversion, and duration calculations. Preserve meaningful legacy semantics and unknown metadata during round trips.
- Make timezone display explicit. Handle UTC, local zones, daylight-saving transitions, zero-duration events, overlapping intervals, open-ended intervals when supported, malformed dates, and very long time ranges. Never silently reinterpret ambiguous timestamps.
- Distinguish read-only and writable data sources. Preserve supported create, read, update, and delete operations and their permissions. Validate edits consistently, persist through the proper source, surface failures, and prevent silent data loss or stale responses overwriting newer state.

Scope discipline: implement the existing product and the requested improvements. Verify whether scheduling dependencies, progress percentages, recurrence, or other project-management concepts actually exist before including them. Do not substitute an unrelated generic Gantt application.

READY-TO-USE PROMPT / 03

A clearer, more interactive UI

Make common actions easy to discover and keep the timeline readable, responsive, and useful with real data.

4. Improve the complete user experience

- Create a coherent interface with clear navigation, readable typography, consistent spacing, visible selection, accessible colors, and helpful labels. Prioritize everyday tasks: loading data, finding an event, navigating time, inspecting details, filtering, and switching views.
- Provide intuitive zoom and pan, sensible time-scale changes, a fit-to-data action, navigation to a selected event, and a visible current-time indicator where meaningful. Start with a useful viewport and keep navigation stable when resizing or changing views.
- Preserve verified grouping, lanes or bands, event styling, legends, configuration, multiple timelines, and other discovered controls. Improve dense and overlapping event layouts, label readability, and selection hit areas without hiding information silently.
- Provide useful event details through a consistent inspector, panel, or dialog. Implement drag, resize, inline editing, and other mutations only where the domain and data source support them. Offer precise form or keyboard alternatives to pointer gestures.
- Make loading, empty, invalid-data, disconnected, and error states explicit and actionable. Preserve useful content during refreshes. Avoid unexplained blank screens, unstable layouts, and controls that appear to work but have no implemented behavior.

5. Support the devices and interaction methods people use

- Adapt the interface to desktop, tablet, and smaller screens. On narrow screens, use a usable single-view layout and deliberate horizontal scrolling where needed. Ensure touch gestures do not conflict with ordinary page navigation.
- Support keyboard navigation, clear focus indicators, accessible control names, logical focus management, and screen-reader access to event details and data. Target WCAG 2.2 AA for applicable UI criteria and report the checks performed.
- Use color together with text or symbols to communicate status. Keep contrast and text legibility sufficient. Respect reduced-motion preferences and provide accessible alternatives for any canvas or WebGL interaction.
- Preserve sensible user preferences, including display timezone, chosen view, and table columns. Explain which settings survive refresh or are shared in links. Keep implementation details out of ordinary product flows.

UI acceptance: demonstrate the main user journeys with screenshots and browser checks. Verify realistic populated screens as well as empty, loading, and failure states; a polished screenshot alone is not evidence of a working interface.

READY-TO-USE PROMPT / 04

One dataset, two synchronized views

The tabular view is a full working representation of the timeline data, with consistent meaning and behavior.

6. Build the table and synchronization contract

Provide **Timeline** and **Table** modes with an obvious view switch. Add a resizable side-by-side mode when screen space permits. Both views must read the same canonical records and use the same validation and data operations.

Concern	Required behavior
Record identity	A table row and its timeline event refer to the same stable ID. Show matching totals for the same filter and scope.
Search and filters	Apply shared search, data-source, group, and date filters. Distinguish the viewport from the data filter; make any visible-range-only table mode explicit.
Selection and navigation	Selecting an event selects its row; selecting a row identifies its event. Provide a reliable reveal-in-timeline action without unexpected viewport jumps.
Edits and deletion	Where supported, either view invokes the same operation. Update both views without a page reload, show save status, and reconcile or roll back failed changes.
Live data and refresh	Apply incoming changes once to the shared model. Handle duplicates, stale responses, reconnects, and deleted records consistently.
Switching and sorting	Preserve filters, selection, and useful navigation state. Table sorting must not unintentionally reorder timeline lanes or reset the time range.

7. Make the table useful for inspection and work

- Derive columns from the verified schema. Include the record identifier, title or label, start, end, and duration when meaningful; expose actual groups, status, source, and metadata when available. Clearly mark missing or non-applicable values.
- Provide typed sorting, search, filtering, configurable columns, column resizing, readable date formatting, and record details. Use pagination or row virtualization when measurements justify it. Define stable ordering and selection behavior across pages or unloaded rows.
- Expose supported editing actions with the same permissions, validation, and error messages as the timeline. Export supported formats consistently; if CSV export is added, document its scope and handle delimiters, Unicode, dates, and spreadsheet formula injection safely.

Synchronization acceptance: test the same actions from both views and verify record IDs, values, counts, persistence, and error recovery. Visual similarity alone does not establish equivalence.

Performance and resilience

Make optimization measurable and preserve correct behavior under load, failures, and repeated use.

8. Establish and meet performance budgets

- Measure the original application before optimizing. Define small, typical, and stress datasets based on expected use, including dense time windows, long labels, many groups, and sustained updates when supported. Use reproducible fixtures and fixed random seeds.
- Record hardware, operating system, browser and runtime versions, build mode, viewport, network conditions, dataset sizes, cache state, and benchmark procedure. Compare equivalent workflows and data in the original and rebuilt applications.
- Measure startup and data-ready time, rendering and update latency, pan and zoom responsiveness, search and filtering latency, table scrolling, memory use, and API or persistence performance where applicable. Use repeated runs and report distributions or median and tail latency where meaningful.
- Set numerical acceptance budgets after establishing the baseline and before evaluating the final implementation. State each target, measurement method, and rationale. Report before-and-after results and tradeoffs; never describe performance gains that were not measured.
- Profile first, then address demonstrated bottlenecks. Consider viewport culling, table virtualization, incremental rendering, indexing, batching, caching, request cancellation, and background computation as needed. Avoid redundant full renders and unbounded caches.
- Check repeated loading, filtering, switching, resizing, and reconnecting for resource leaks. Clean up event listeners, timers, workers, subscriptions, and renderer or GPU resources where used. Keep the interface responsive and explain limits when data exceeds the supported capacity.

9. Handle failure and security at real boundaries

- Validate external data and API input. Safely render untrusted labels, descriptions, URLs, and imported content. Protect supported write operations and preserve existing authentication and authorization boundaries. Keep credentials out of source code, client bundles, and logs.
- Handle network timeouts, malformed responses, partial failures, cancellation, source unavailability, and reconnection. If the application streams updates, define ordering, deduplication, retry, and resynchronization behavior for the actual protocol.
- Document dependency and configuration risks, correct confirmed issues, and use suitable automated checks. Provide useful diagnostics without exposing sensitive data. Make build, startup, configuration, shutdown, and recovery behavior reproducible.

Evidence rule: if the original cannot run, state that a direct comparison is unavailable. Benchmark the replacement against declared budgets and clearly label the limitation; do not invent a baseline.

Testing from end to end

Prove the important behaviors and failure paths with reproducible tests, realistic data, and honest results.

10. Build a layered verification strategy

- **Unit tests:** verify date parsing and conversion, interval logic, sorting, filtering, data normalization, validation, state transitions, and view selectors. Cover invalid inputs, missing fields, duplicate IDs, boundary dates, leap years, daylight-saving transitions, and zero-duration or overlapping events.
- **Integration and contract tests:** verify data loading, adapters, persistence, API formats, configuration, and live updates where present. Test round trips with existing datasets and representative unknown metadata. Exercise timeout, failure, retry, conflict, and reconnection behavior.
- **End-to-end browser tests:** cover clean startup, data loading, search, filters, zoom, pan, details, view switching, synchronized selection, supported edits and deletion, save and reload, and export where available. Verify outcomes against canonical data rather than screenshots alone.
- **Regression and visual tests:** map preserved features and fixed defects to repeatable checks. Test long labels, crowded intervals, many groups, extreme zoom levels, resizing, and representative screen widths. Control fonts, clocks, and test data for stable visual comparisons.
- **Accessibility and compatibility checks:** combine automated checks with keyboard, focus, touch, and screen-reader review where available. Exercise Chrome, Edge, Firefox, and Safari or the available browser engines; record exactly what ran and any actual-browser or device gaps.
- **Performance and robustness tests:** automate the agreed benchmarks and test repeated interactions, large datasets, streaming bursts where relevant, and resource cleanup. Test actual input and permission boundaries rather than relying only on dependency scans.

11. Make every quality claim auditable

- Create a requirements-to-tests matrix covering every preserved feature, requested enhancement, critical workflow, and important failure path. Use meaningful assertions; do not add superficial tests merely to increase a coverage percentage.
- Run formatting, linting, applicable type or static checks, unit tests, integration tests, end-to-end tests, and the production build through documented commands and CI. Verify a clean installation and launch using the README instructions.
- Fix discovered defects and rerun affected checks. Record exact commands, environment, test counts, results, logs or report locations, benchmark results, and manual checks. Distinguish passed, failed, skipped, blocked, and not-run items.

Completion standard: do not claim exhaustive correctness, zero defects, or successful tests without evidence. Critical workflows must pass. Unresolved failures and untested areas must remain visible, with their impact and reproduction steps.

READY-TO-USE PROMPT / 07

Documentation and final delivery

Deliver an application another developer can install, understand, verify, and maintain.

12. Write practical documentation in Markdown

Keep README.md short and suitable for a newcomer. Include the purpose, prerequisites and versions, installation and startup commands, local URL, sample data, basic timeline and table use, test commands, and links to detailed guides. Verify the instructions from a clean checkout.

Provide these Markdown documents, combining files only when it improves clarity:

Document	Required content
docs/analysis.md	Audit scope, commit SHA, source inventory, existing behavior, defects, assumptions, and exclusions.
docs/architecture.md	Modules, data flow, shared state, rendering, and justified design decisions.
docs/data-model.md docs/api.md	Schemas, time semantics, validation, compatibility, and actual integration contracts; mark API documentation non-applicable if appropriate.
docs/user-guide.md docs/configuration.md	Timeline and table workflows, keyboard controls, settings, data sources, and worked examples.
docs/development.md docs/testing.md	Project structure, build workflow, fixtures, test commands, results, and known verification gaps.
docs/performance.md docs/traceability.md	Reproducible benchmarks and comparisons; requirements, implementation, and verification mapping.
docs/deployment.md docs/migration.md docs/troubleshooting.md	Installation, deployment, rollback, compatibility or migration, common errors, and known limitations.

Maintain CHANGELOG.md and contribution guidance. Preserve license and attribution notices. Link native schemas, API specifications, and generated test reports from Markdown. Use diagrams and screenshots where helpful.

13. Deliver and assess the finished work

Deliver runnable source, build and configuration files, sample data, tests and results, benchmarks, migration support where required, and Markdown documentation. Summarize preserved features, improvements, measured gains, and remaining limitations with evidence links.

Accept completion only when clean setup works, the feature matrix is satisfied, both views stay consistent, supported data survives load/edit/save, critical tests pass, performance meets declared budgets or misses are disclosed, and documentation matches the implementation. Leave no placeholders in required workflows. If blocked, deliver completed work and explain what remains and why.

The legacy appearance to preserve

Original project screenshots, reproduced for visual comparison. They show two configured presentations of OpenBEXI Timeline.

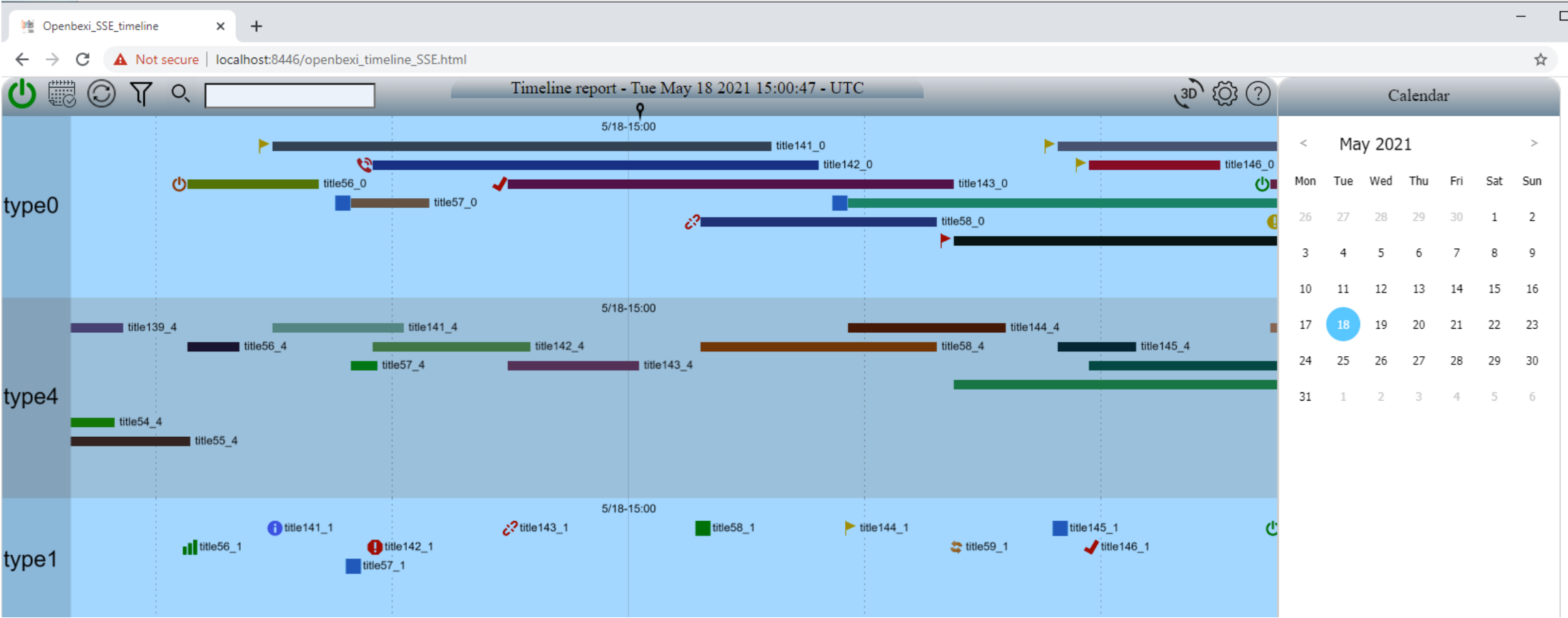


Figure 1 · Primary reference

1. Compact metallic toolbar.
2. Broad blue type bands.
3. Thin duration bars and small glyphs.
4. Darker left type gutter.
5. Central reference marker.
6. White calendar on the right.

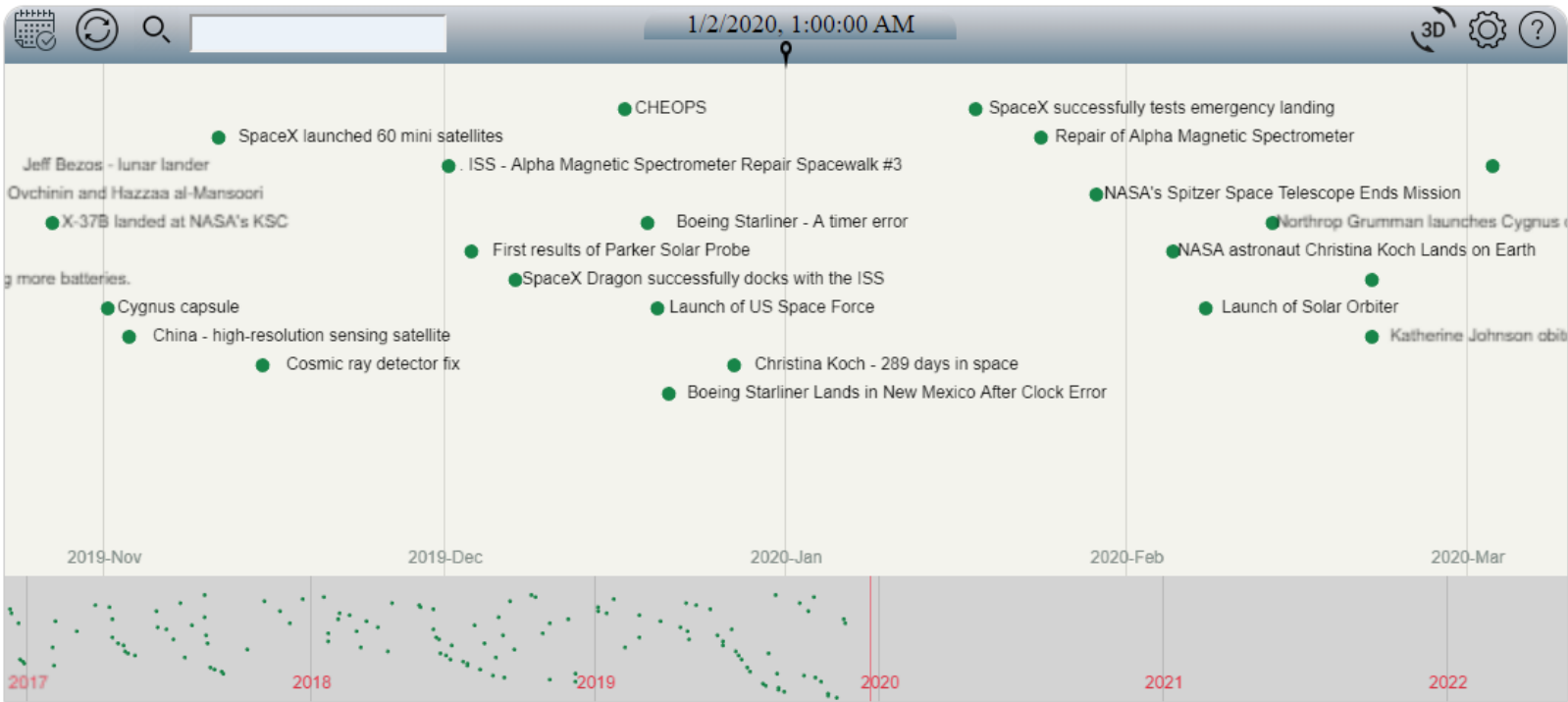


Figure 2 · Alternate legacy configuration

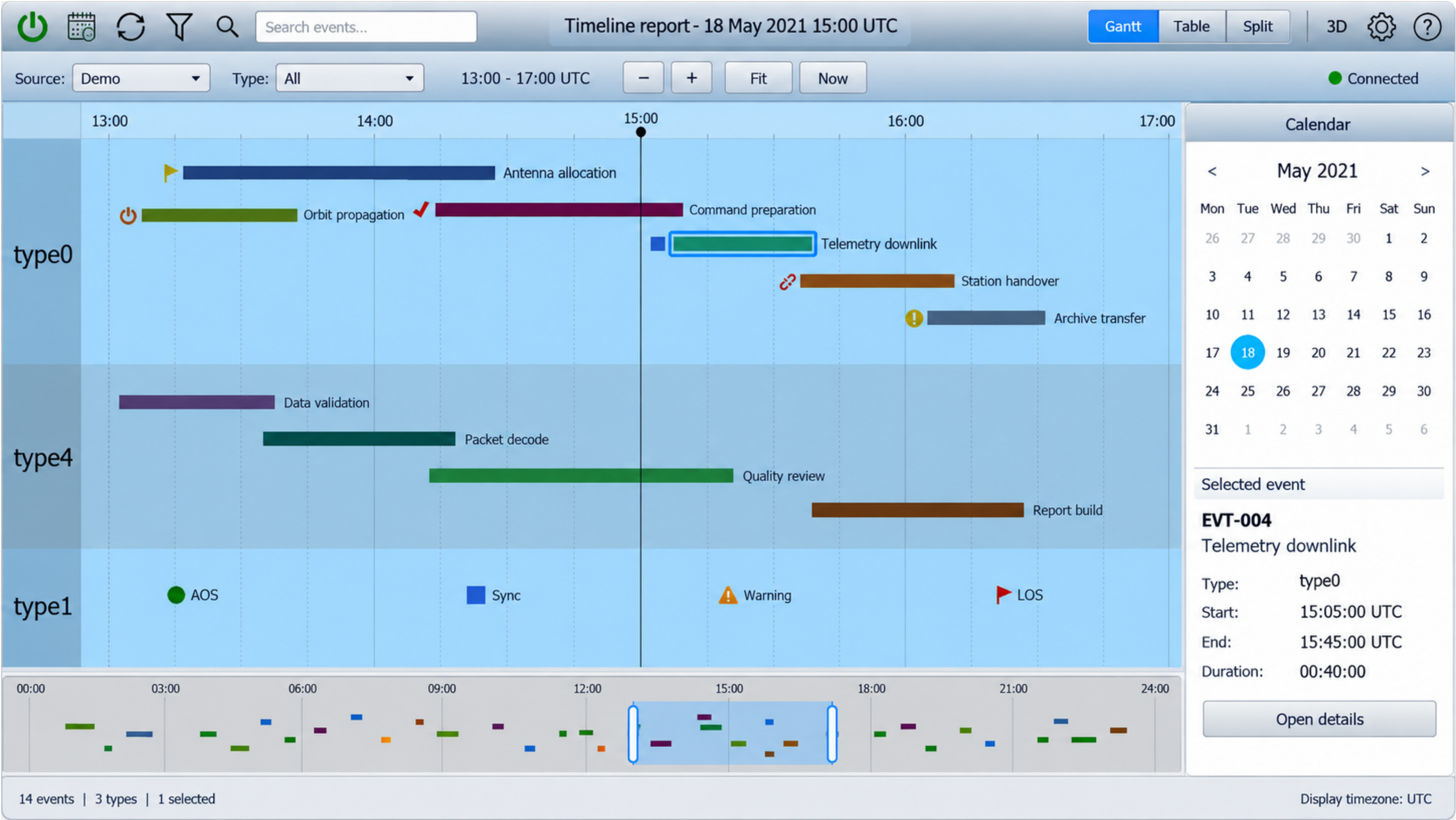
Neutral canvas, small point markers, month ticks, and a bottom overview. Preserve this configurable presentation when the original supports it. The rebuilt grouped view adopts the overview while keeping the blue-band identity from Figure 1.

Visual intent: recognizable continuity, sharper text, consistent spacing, and a separate table mode.

Sources: repository doc/openbexi_timeline_example.PNG and doc/openbexi_timeline_space_exploration.PNG. Existing OpenBEXI artwork and attribution retained.

Gantt timeline - close to the legacy UI

Retain the silver toolbar, blue grouped bands, thin event bars, reference marker, calendar, and overview. New controls remain compact.

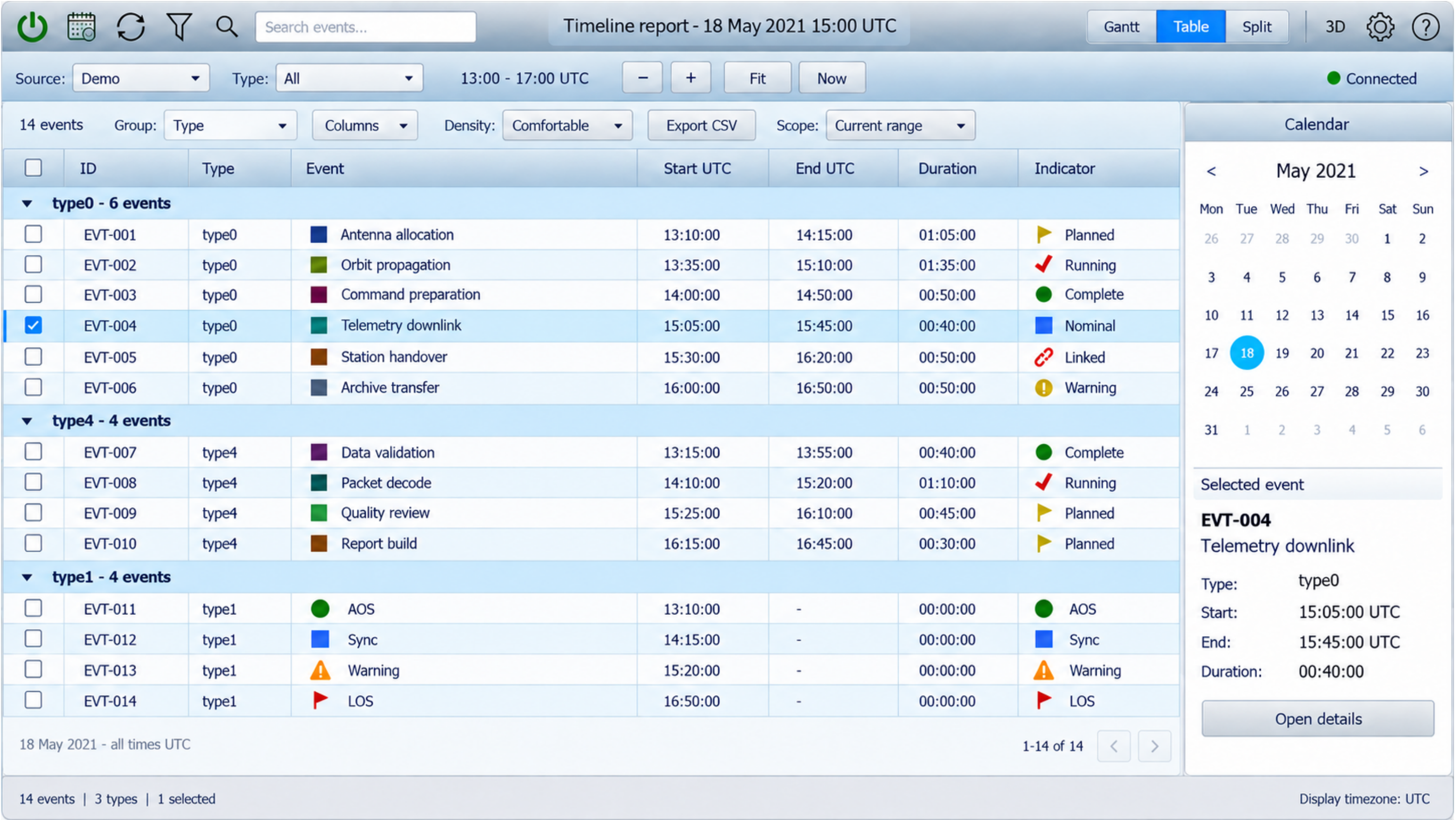


Read the picture: type bands hold staggered events; the selected teal bar remains its original color; the right pane shows the same EVT-004 record; the overview controls the time viewport.

Proposed mockup, not a screenshot of implemented software. Use the exact visual contract and fixture for dates, record count, and bar geometry.

Tabular timeline - the same application

The shared toolbar, filters, calendar, selection, and timezone stay in place. Grouped rows replace the central timeline canvas.



Read the picture: type headers mirror timeline groups; color swatches preserve event identity; aligned time columns support comparison; the selected row and right inspector both identify EVT-004.

Proposed mockup. Indicator text is illustrative metadata; selection is a separate UI state. See the exact column widths and fixture in the visual contract.

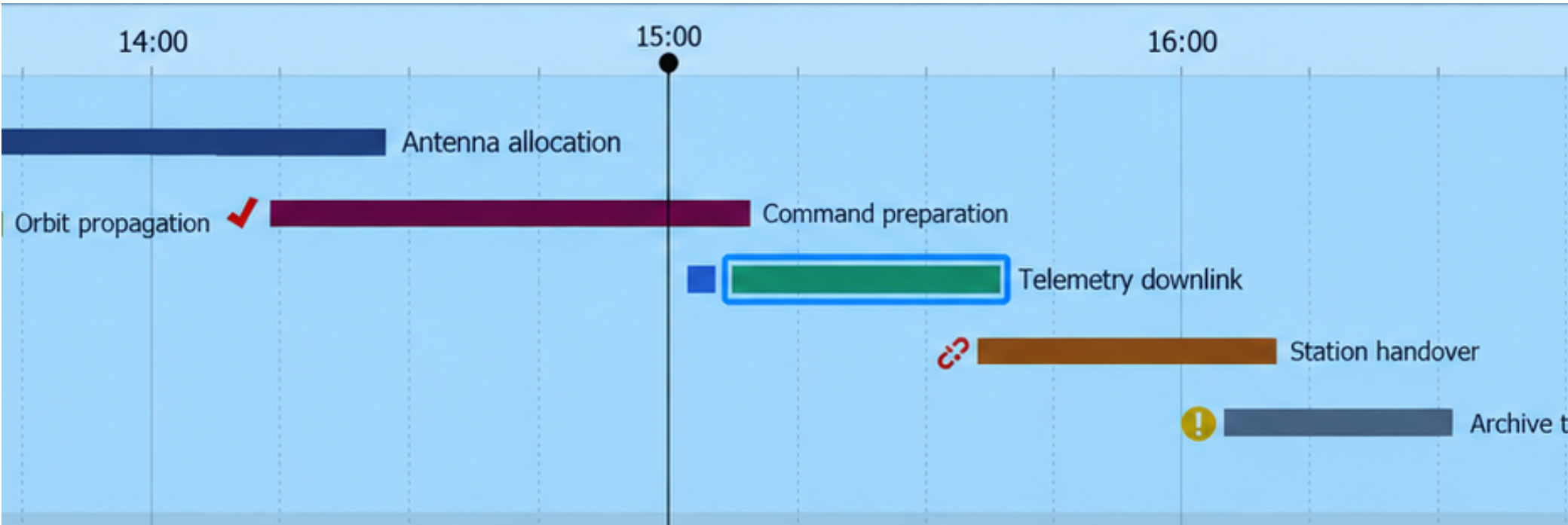
Selection, legibility, and shared controls

These enlarged views show the required treatment of the selected event, shared shell, and exact data-oriented table styling.

A • Compact common toolbar - familiar icons, centered report time, and visible Gantt / Table / Split selector.



B • Timeline event selection - preserve the teal fill, add a thin blue outline, and keep the label outside the bar.



Shared inspector

Stable record ID.
Full title and type.
Explicit UTC times.
Calculated duration.

The event and row must both point to the same record. Hover, keyboard focus, and selection remain distinct states.

C • Table selection - checked box, pale cyan row, blue left rule, stable ID, readable time columns, and status kept separate.

☐	ID	Type	Event	Start UTC	End UTC	Duration	Indicator
▼ type0 - 6 events							
☐	EVT-001	type0	■ Antenna allocation	13:10:00	14:15:00	01:05:00	🚩 Planned
☐	EVT-002	type0	■ Orbit propagation	13:35:00	15:10:00	01:35:00	✓ Running
☐	EVT-003	type0	■ Command preparation	14:00:00	14:50:00	00:50:00	● Complete
☒	EVT-004	type0	■ Telemetry downlink	15:05:00	15:45:00	00:40:00	■ Nominal

Enlargements of Figures 3 and 4. Exact pixel dimensions, colors, and interactions are specified in the following pages.

Legacy appearance is a requirement

Implement the visual contract below as part of the original rebuild brief. It governs the appearance of both views.

14. Preserve the recognizable OpenBEXI interface

- Use the repository screenshots as the legacy reference: `doc/openbexi_timeline_example.PNG` for the blue grouped timeline and `doc/openbexi_timeline_space_exploration.PNG` for the neutral event timeline with overview. Inspect the active model and CSS before implementing. Preserve both configurations when supported; the blue grouped view is the default visual target in these mockups.
- Retain the compact silver-to-blue-gray toolbar, icon-driven controls, continuous time canvas, alternating blue bands, darker type-label gutter, thin colored duration bars, small event glyphs, central time marker, optional right calendar, and overview. Keep existing model-driven colors and glyph meanings. Improve readability and accessibility without replacing this composition with a generic project-management dashboard.
- Keep the time axis horizontal and `type0`, `type4`, `type1` as vertically stacked example bands. Allow several staggered event tracks within a band. Do not turn the Gantt into a spreadsheet with one permanent task row per event; that presentation belongs to the separate Table view.
- Keep controls in the legacy order: connection, calendar, refresh, filter, search; report time in the center; view controls, 2D/3D where supported, settings, and help at the right. Add a compact Gantt / Table / Split selector. Keep the selected view visible without adding a tall branding header or permanent navigation sidebar.

15. Read the figures correctly

Figures 1 and 2 are original repository screenshots. Figures 3 and 4 are proposed visual mockups, generated from those references; Figure 5 enlarges their details. The exact geometry, sample records, and behavior specified here are authoritative where illustrative pixels differ. Do not reproduce accidental spacing, date placement, or decorative overview marks as data. Render every event from its actual timestamps.

The satellite-operation names, IDs, and indicator text in the mockups are illustrative acceptance-test data, not claims about the existing schema. Adapt the fixture through the application's actual model. Selection is UI state and must never overwrite an event's status or icon metadata.

Source references

[Blue grouped timeline screenshot](#) · [Space-exploration screenshot](#) · [Classic CSS](#) · [Modern CSS](#).

Recheck these references at the recorded implementation commit.

READY-TO-USE PROMPT / 09

Exact Gantt layout

Use a 1600 x 900 CSS-pixel desktop reference viewport at 100% zoom. Coordinates are measured from the top-left of the app.

Region	Position and size
Main toolbar	x=0, y=0, w=1600, h=56. Gradient #D3D8DF to #6C8A9C. Controls use 40 x 40 hit areas, approximately 24-28 px icons.
Filter strip	x=0, y=56, w=1600, h=56. Source, Type, visible range, zoom, Fit, Now, connection state.
Timeline and calendar	Canvas x=0, y=112, w=1304, h=620. Calendar/inspector x=1304, y=112, w=296, h=740.
Time ruler and gutter	Ruler y=112..152. Fixed type gutter x=0..88. Plot spans x=100..1292 after internal padding.
Three example bands	type0: y=152..400; type4: y=400..604; type1: y=604..732. Default fills alternate #A6D7F8 / #99BFD3; gutter #82A9BF.
Overview and footer	Overview x=0, y=732, w=1304, h=120. App footer x=0, y=852, w=1600, h=48.

16. Render time and events precisely

- In the example, show 18 May 2021, 13:00-17:00 UTC. Major hour ticks are x=100, 398, 696, 994, 1292. Minor ticks occur every 15 minutes. The black pin and thin reference line mark 15:00 at x=696. This is a reference/playhead time, not necessarily the live current time; show a separately labeled Now marker when relevant.
- Use $x(t) = 100 + 1192 \times ((t - 13:00) / 240 \text{ minutes})$. Draw durations from x(start) to x(end), clipped to the plot, with 10-12 px bar height, square or 1 px radius corners, and 32 px track pitch. Point events use 12-16 px glyphs, not artificially long bars. Keep a minimum 24 px interaction target without enlarging the visible mark.
- Use 14 px body and event text, 13 px tick labels, and 22 px type labels. Position event labels 8 px after the bar when space permits. Resolve collisions with additional tracks or label placement; ellipsize only as a last resort and expose the full text on focus and in the inspector. Never paint labels across the calendar.
- Retain each event's original fill and icon on selection; add a 2 px #168BFF outline with a 2 px gap. EVT-004 spans x=720.83..919.50. Type bands resize with content, preserving group order; overflow scrolls rather than hiding events. The 1600 x 900 values define the example, not hardcoded limits for all datasets.

Exact tabular layout

Use the same shell, calendar, dataset, timezone, search state, and selection. Only the central view changes.

Element	Required appearance
Shared shell	Keep the toolbar, filter strip, x=1304 calendar boundary, and footer at identical coordinates. Highlight Table in the view selector. Hide the Gantt overview in full Table mode.
Table controls	x=0, y=112, w=1304, h=52. Show record count, Group: Type, Columns, Density, Export CSV where implemented, and Scope: Current range.
Header and groups	Column header y=164..208 (44 px). Group headers are 32 px, pale blue #C8E8FA, bold 14 px text, chevron, group name, and record count.
Rows and pagination	Comfortable rows: 34 px. Compact: 28 px. With 14 records, group sections end at y=780; pagination fills y=780..852. Both headers and rows align precisely.
Colors and selection	Alternate #FFFFFF and #F0F8FD rows; 1 px #C8DDEA grid. Selected row #DCF1FF with 3 px #168BFF left rule and checked box. Hover #E8F5FF.

17. Default column widths at 1600 px

Checkbox 44 · ID 112 · Type 92 · Event 440 · Start UTC 148 · End UTC 148 · Duration 132 · Indicator 188 = 1304 px. Keep checkbox and ID sticky during horizontal scrolling. Allow column resize, reorder, and hide, with a restore-default action. Use tabular numerals for dates and durations; align time columns consistently.

- Use 14 px row text, medium-weight headers, and 10 px cell padding. Prefix event titles with their actual color/glyph. Show textual indicator meaning beside the icon; do not rely on color alone. Selection and keyboard focus are distinct states. Use a visible 2 px focus ring around the focused cell.
- Group initially in legacy order type0 (6), type4 (4), type1 (4); sort by start ascending within each group, breaking ties by ID. Collapsing a group retains its selection and count. Offer an explicit ungrouped mode. The canonical fixture order may change under this sort, but membership and values must remain identical.
- For the single-day fixture display HH:mm:ss and a visible date caption, 18 May 2021 - all times UTC. For multiple dates use YYYY-MM-DD HH:mm:ss with wider columns or horizontal scroll. Point-event End displays a dash and Duration 00:00:00. Unknown duration displays a dash, never a fabricated zero.
- Use one row per canonical record. Confirm that the IDs counted in the table match the filtered timeline records, independently of virtualized rows or the number of bars currently visible. Keep the selected EVT-004 row and calendar inspector consistent when switching views.

Details, states, and visual acceptance

Define the behavior behind every visible element and verify the legacy resemblance with comparable screenshots.

18. Calendar, selection, and navigation

- Default right pane: 296 px wide, white background, silver Calendar header, month navigation, Monday-first day grid, and cyan #27B8F5 selected-day circle. For the fixture, show May 2021 and select the 18th. Today uses a separate outline. Keyboard and screen readers must expose these meanings.
- Below the calendar, show Selected event, ID, complete title, type, start, end, duration, and Open details. Selecting an item updates this panel from the shared record. Selecting a calendar day navigates time while retaining the window span. Do not silently change the selected record; indicate when it lies outside the current range.
- Open details displays a 360 px right drawer or accessible dialog with a close button, full values, metadata, and edit actions only for writable sources. Escape closes and restores focus. Failed saves retain the user's input, show the error beside the field/action, and keep both views on the last confirmed data.
- Scope: Current range applies the same interval predicate in both views. For half-open intervals use $\text{start} < \text{windowEnd}$ AND $\text{end} > \text{windowStart}$; points satisfy $\text{windowStart} \leq \text{time} < \text{windowEnd}$. Preserve different legacy boundary semantics when required and document them. Scope: All matching retains filters but ignores the visible range, with the different total clearly labeled.

19. Responsive layout and operational states

- At widths 1200-1599 px, keep the main layout flexible and allow toolbar controls to move into an overflow menu. Below 1200 px, collapse the calendar into a drawer. Below 768 px, use a compact toolbar, a Filters popover, and full-width Gantt/Table tabs; keep 44 px touch targets and deliberate horizontal table scrolling.
- Split mode at 1440 px or wider hides the calendar by default and divides the content into 60% Gantt / 40% Table with a 6 px resizable divider. At 768-1439 px stack the views vertically, Gantt above Table. Below 768 px use the view tabs. Keep search, filters, timezone, selection, and record changes synchronized in every arrangement.
- Loading: retain the shell and show labeled progress. Empty: keep the ruler or headers and show No events in this range with Reset filters. Disconnected: retain the last data, mark it Stale with its last-successful time, and offer Retry. Saving and validation errors must be visible and keyboard accessible.

Visual acceptance and handoff

Make visual fidelity, consistent data, and useful interaction independently reviewable.

20. Visual tests and handoff

Use the final-page fixture and a fixed clock. Capture original and replacement with the same model, viewport, timezone, and data. Check toolbar order, band structure, density, calendar position, colors, glyphs, bar coordinates, selected state, table widths, and text clipping. At the 1600 x 900 target, allow at most 2 CSS px difference from specified static geometry; allow font antialiasing differences. Repeat at 1280, 1024, and 390 px widths. Validate meaning, focus order, contrast, and data equality separately from screenshots.

Document tokens and component rules in docs/ui-spec.md; preserve reference screenshots and the exact fixture with the tests. Explain any justified departures from the visual contract. The original analysis, compatibility, performance, testing, and Markdown requirements remain in force.

Required screenshot and interaction checklist

- **Default Gantt:** all three groups visible, legacy toolbar ordering, reference marker at 15:00, no clipped controls, and correct duration positions. Confirm point events remain points and the overview contains only records actually loaded in its declared scope.
- **Selected Gantt and Table:** highlight EVT-004 in each view, retain its teal color and Nominal indicator, show the same inspector values, and preserve the selected ID through switching, sorting, filtering, and virtualized scrolling.
- **Expanded details:** verify the full title and timestamps, keyboard focus trap where appropriate, Escape behavior, read-only state, edit validation, successful save, and failed-save recovery. No source metadata may be replaced by selection state.
- **Grouped and ungrouped table:** verify the 6 / 4 / 4 group totals, ascending-time sort within each group, stable tie-breaking, column alignment, resized widths, sticky ID, collapsed groups, and the restored default layout.
- **Responsive and failure states:** capture calendar open and closed, split view, narrow-screen overflow, loading, empty results, disconnected stale data, and a validation error. Confirm toolbar labels and data remain reachable without overlapping or disappearing controls.

What the developer must hand over

Include docs/ui-spec.md with the implemented design tokens, component dimensions, responsive rules, and justified differences from the figures. Include baseline and updated screenshots, the deterministic fixture and loader adapter, automated visual-test commands and results, and a brief manual accessibility report. Map each visual requirement to its component and verification evidence. Report remaining mismatches explicitly.

One exact fixture for both views

14 synthetic records · 18 May 2021 · UTC · reference time 15:00 · visible range 13:00-17:00 · initially select EVT-004

ID	Type	Event	Start UTC	End UTC	Duration	Indicator	Event color
EVT-001	type0	Antenna allocation	13:10:00	14:15:00	01:05:00	Planned	#253C78
EVT-002	type0	Orbit propagation	13:35:00	15:10:00	01:35:00	Running	#4C7900
EVT-003	type0	Command preparation	14:00:00	14:50:00	00:50:00	Complete	#601654
EVT-004	type0	Telemetry downlink	15:05:00	15:45:00	00:40:00	Nominal	#008A80
EVT-005	type0	Station handover	15:30:00	16:20:00	00:50:00	Linked	#986015
EVT-006	type0	Archive transfer	16:00:00	16:50:00	00:50:00	Warning	#486A85
EVT-007	type4	Data validation	13:15:00	13:55:00	00:40:00	Complete	#674E8F
EVT-008	type4	Packet decode	14:10:00	15:20:00	01:10:00	Running	#006969
EVT-009	type4	Quality review	15:25:00	16:10:00	00:45:00	Planned	#16833F
EVT-010	type4	Report build	16:15:00	16:45:00	00:30:00	Planned	#8A511C
EVT-011	type1	AOS	13:10:00	-	00:00:00	AOS	#008535
EVT-012	type1	Sync	14:15:00	-	00:00:00	Sync	#2056BC
EVT-013	type1	Warning	15:20:00	-	00:00:00	Warning	#E79915
EVT-014	type1	LOS	16:50:00	-	00:00:00	LOS	#C82620

Fixture rules. End is absent for the four point events; their duration is zero because they are explicitly points. Keep the fixture's color mapping identical in both views. Persisted metadata determines glyphs; the screenshot glyphs are examples. Use one canonical record per ID, and include exactly these 14 records in the fixture overview.

Acceptance sequence. Load this fixture; confirm 6 / 4 / 4 group counts. Select EVT-004 in Gantt; switch to Table and verify the same record, values, and inspector. Change its end to 16:00 through a writable test adapter: duration becomes 00:55:00 and the bar endpoint becomes x=994 at the reference viewport. Reload and confirm persistence. Restore the original fixture after the test.

END OF UPDATED PROMPT. Deliver the rebuilt application against both the original requirements and this visual contract. The figures describe the target appearance; they do not claim that implementation or testing has already been completed.